



Money Manager Application



Version: 1.0

Author: Zaid

Language: Java (JDK 17+)

Architecture: 3-Tier + SOLID

Date: May 2026

Status: Draft

Table of Contents

1	Introduction
2	Project Objectives
3	System Context Diagram
4	Use Case Diagram
5	Functional Requirements
6	Non-Functional Requirements
7	System Architecture
8	UML Class Diagrams
9	I/O Data Flow Diagram
10	Database Design
11	OOP & Array Design Rationale
12	User Interface Specification
13	Future Enhancements
14	Glossary
15	Revision History

1 Introduction

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the **Money Manager Application**, a desktop financial management tool built in **Java (JDK 17+)**. The application enables users to track income and expenses, manage monthly budgets, maintain notes and financial goals, and view summarised reports.

Attribute	Detail
Project Name	Money Manager Application
Programming Language	Java (JDK 17+)
Architecture	3-Tier (Presentation → Service → Repository) + SOLID
Core Focus	Object-Oriented Programming fundamentals & 1D Arrays
Data Storage	SQLite (via JDBC)
Future Plans	Android (Kotlin) and Web (Spring Boot + React)

2 Project Objectives

- Track expenses and income with full CRUD operations.
- Categorise transactions using enumerations and 1D arrays.
- Manage monthly budgets with threshold alerts.
- Provide a dashboard summarising financial data.
- Implement a Notes & Goals system for reminders and objectives.
- Demonstrate core OOP: encapsulation, inheritance, polymorphism, abstraction.
- Use 1D arrays as the primary in-memory collection structure.
- Maintain a flexible 3-Tier architecture for future expansion.

3 System Context Diagram

The context diagram illustrates the system boundary, primary actor (User), external data store (SQLite), and the console/GUI interface.

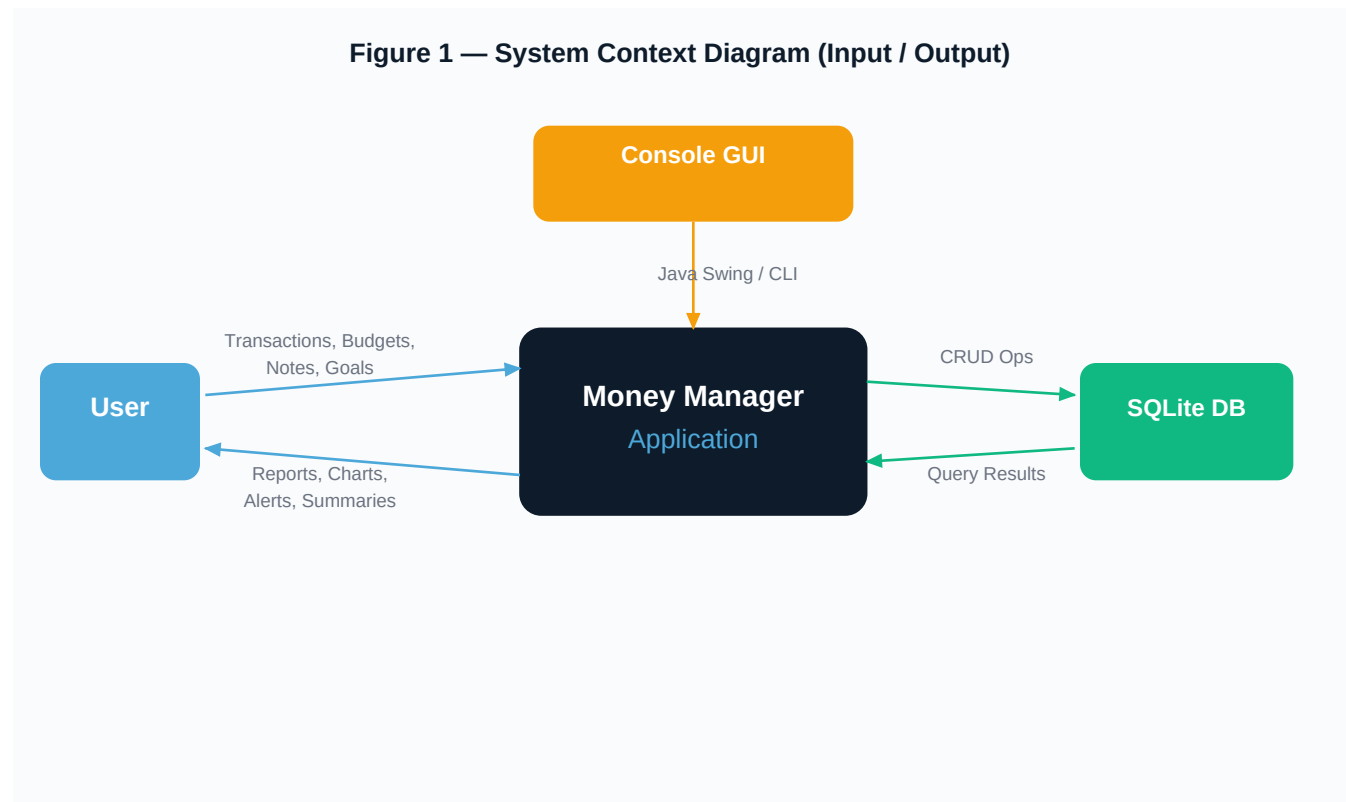


Figure 1 — System Context Diagram showing primary I/O channels.

4 Use Case Diagram

Seven use cases are identified spanning core transaction management through future export.

Figure 2 — Use Case Diagram

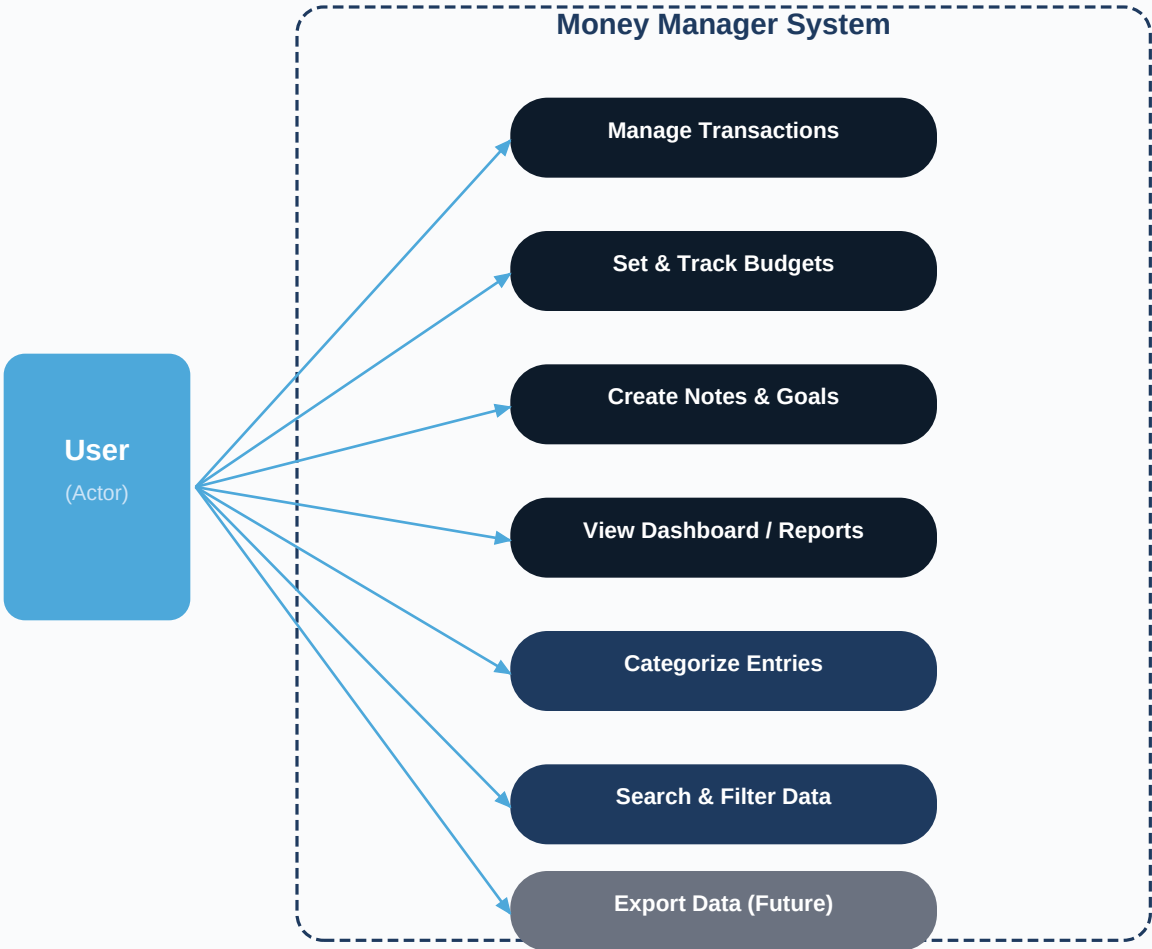


Figure 2 — Use Case Diagram with primary and secondary use cases.

Use Case Descriptions

UC-ID	Use Case	Description	Priority
UC-01	Manage Transactions	Add, edit, delete, list income/expense entries in Transaction[]	High
UC-02	Set & Track Budgets	Create monthly budgets; alert when spending exceeds limits.	High
UC-03	Create Notes & Goals	Record textual notes and financial goals with tags/dates.	Medium
UC-04	View Dashboard	Display totals, category breakdowns, budget utilisation.	High

UC-ID	Use Case	Description	Priority
UC-05	Categorise Entries	Assign categories via Category enum.	Medium
UC-06	Search & Filter	Query by date range, category, or keyword.	Medium
UC-07	Export Data	Future: export reports to PDF / Excel.	Low

5 Functional Requirements

ID	Requirement	Category
FR-01	The system shall allow adding a Transaction with name, amount, category, date.	Transaction
FR-02	Transactions shall be stored in a 1D array (Transaction[]) with dynamic resizing.	Data Structure
FR-03	The system shall support editing and deleting transactions by ID via linear search.	Transaction
FR-04	The system shall allow creation of Budget objects with name and limit amount.	Budget
FR-05	Remaining budget is calculated by iterating Transaction[] and summing per category.	Budget
FR-06	Alert the user when spending reaches 80% or 100% of a budget limit.	Budget
FR-07	CRUD operations on Note objects (title, content, date, tag).	Notes
FR-08	Display a text/Swing dashboard with totals, averages, category summaries.	Dashboard
FR-09	Persist all data to SQLite via JDBC.	Persistence
FR-10	Validate all inputs (non-empty strings, positive amounts, valid dates).	Validation

6 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Array ops complete in $O(n)$ or better for up to 10,000 records.
NFR-02	Usability	No more than 3 clicks/inputs for any core operation.
NFR-03	Reliability	Invalid input handled via try-catch without crashes.
NFR-04	Portability	Runs on any JDK 17+ platform (Windows, macOS, Linux).
NFR-05	Maintainability	SOLID principles with clear 3-tier separation.
NFR-06	Security	Local DB with no network exposure in initial release.
NFR-07	Scalability	Arrays replaceable with ArrayList/HashMap via Dependency Inversion.

7 System Architecture

The application follows a **3-Tier architecture** with SOLID principles.

Figure 3 — Three-Tier Architecture

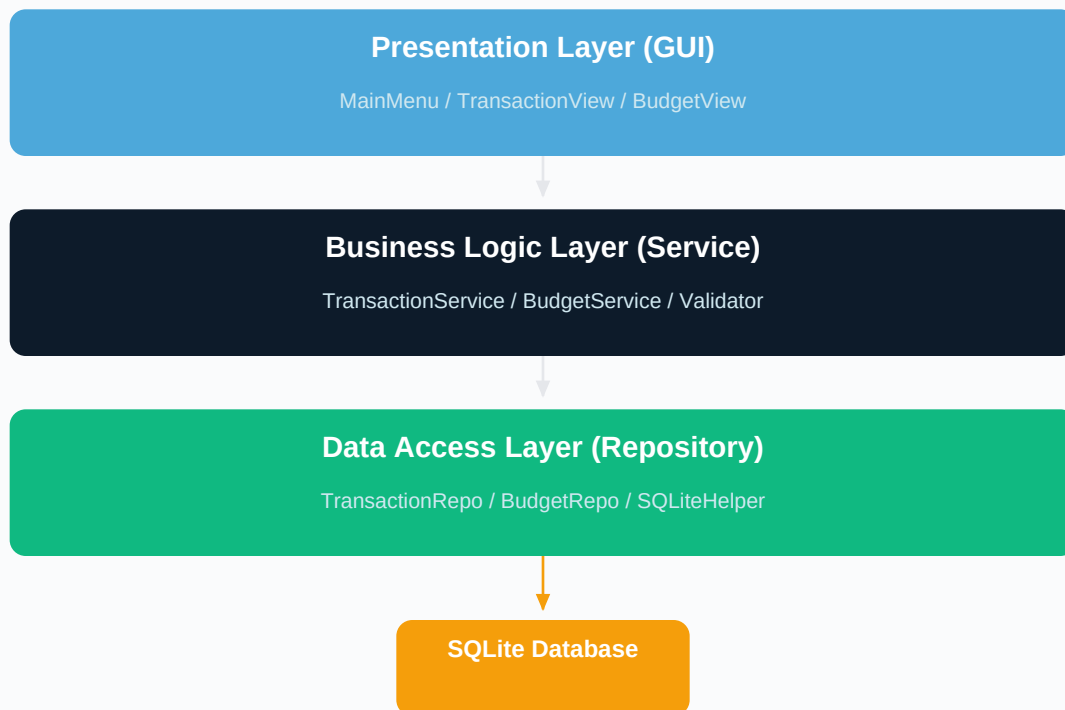


Figure 3 — Three-Tier Architecture with SOLID principle mapping.

SOLID Principles Applied

- **S — Single Responsibility:** Each class has one job. `TransactionService` handles logic; `TransactionRepo` handles persistence.
- **O — Open/Closed:** New features added as new classes without modifying existing services.
- **L — Liskov Substitution:** Any repository implementation can replace another through `IRepository`.
- **I — Interface Segregation:** Small interfaces like `ITransactionRepo` prevent bloated contracts.
- **D — Dependency Inversion:** Services depend on abstractions, not concrete classes.

8 UML Class Diagrams

The following UML class diagrams define every domain model and service class in the Money Manager application. Each model is a fully encapsulated POJO with private fields, public getters/setters, and a `toString()` override. Service classes manage their entities using **1D arrays** as the primary in-memory data structure.

8.1 BaseEntity (Abstract Superclass)

All domain objects extend `BaseEntity`, inheriting a unique `id` and `createdAt` timestamp. This demonstrates **inheritance** and avoids field duplication.

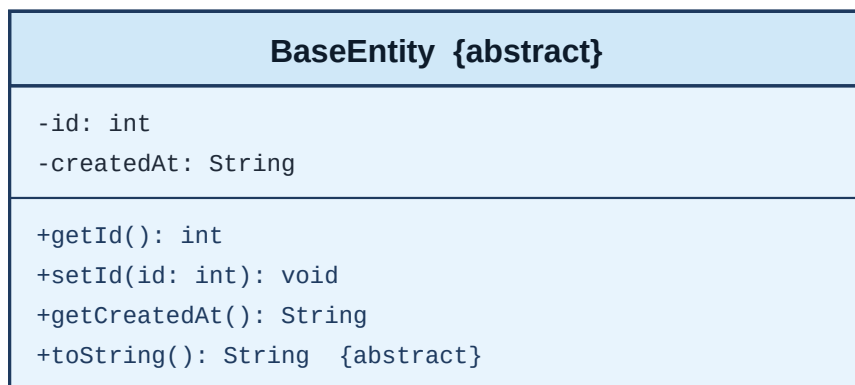


Figure 4a — BaseEntity abstract superclass.

8.2 Transaction

Represents a single income or expense entry. All fields are private with controlled access through getters and setters (**encapsulation**). The category field maps to the `Category` enum.

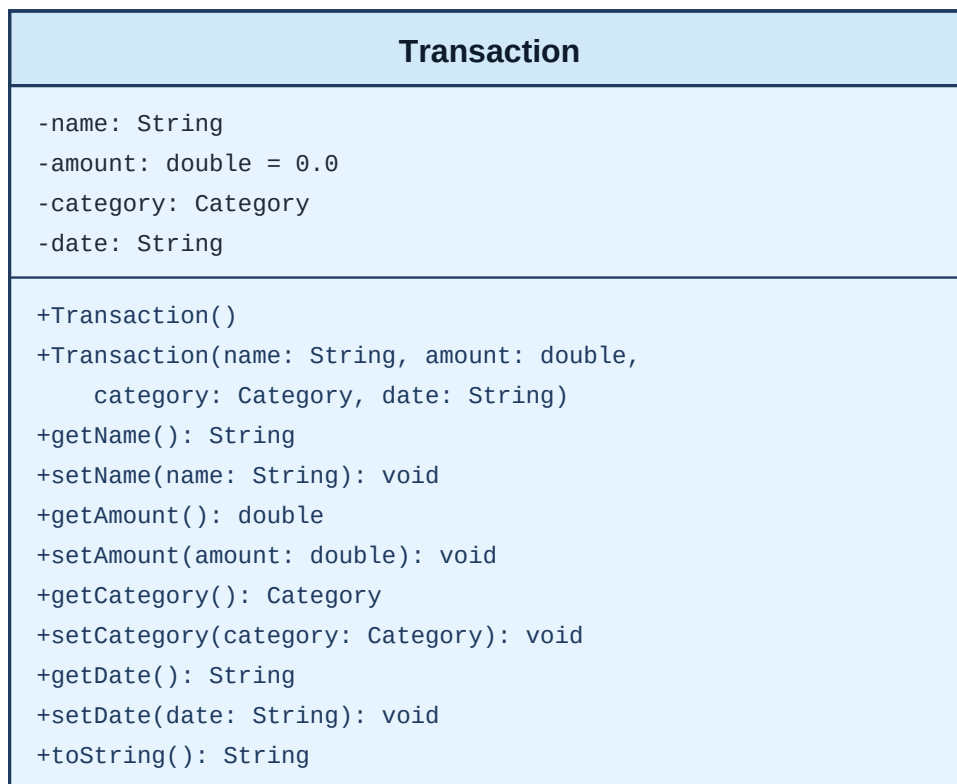


Figure 4b — Transaction class UML diagram.

8.3 Budget

Stores a named spending limit. The service layer compares actual spending against amount to trigger threshold alerts.

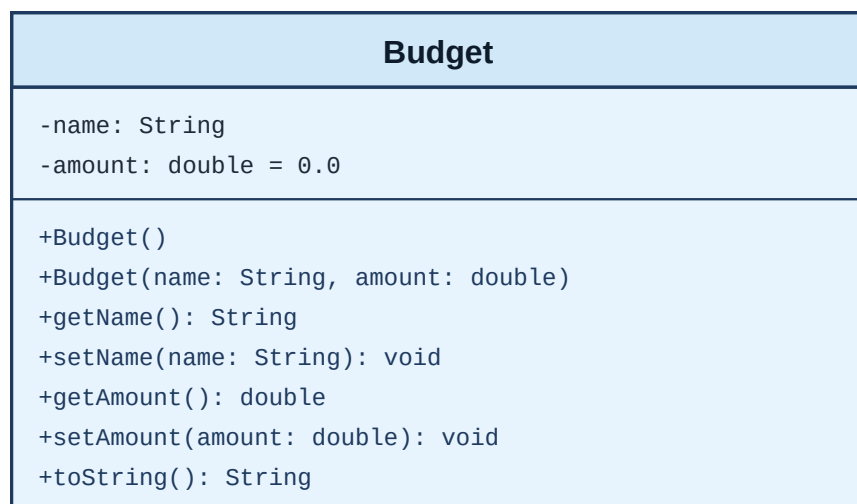


Figure 4c — Budget class UML diagram.

8.4 Note

Represents a textual note or financial goal. The tag field allows categorisation (e.g., 'savings', 'reminder', 'goal').

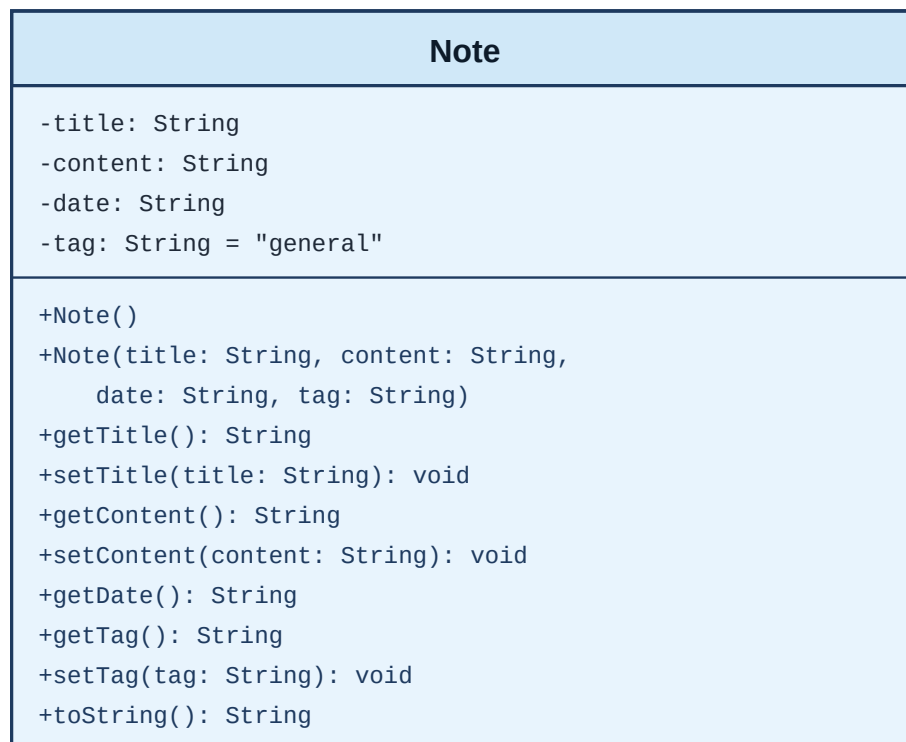


Figure 4d — Note class UML diagram.

8.5 Category (Enumeration)

A Java enum enforcing a fixed set of valid transaction categories. Prevents invalid string inputs and enables type-safe comparisons.

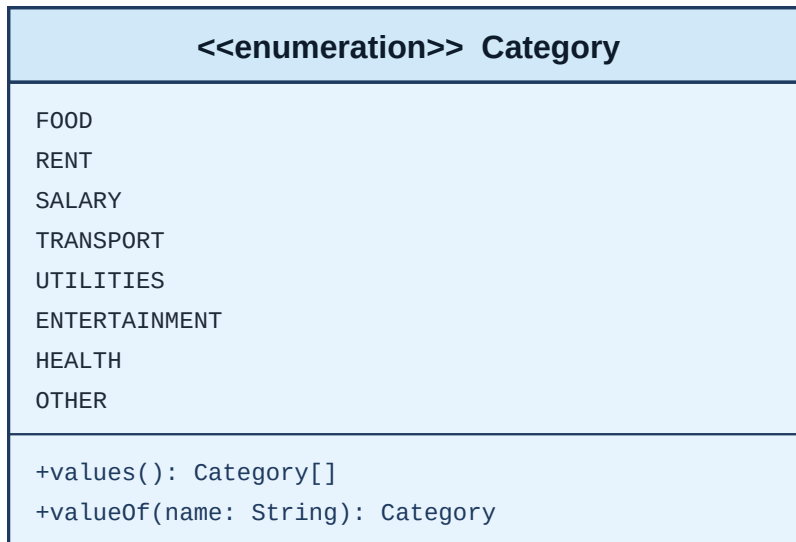


Figure 4e — Category enumeration UML diagram.

8.6 TransactionService

Manages an in-memory Transaction[] array. Demonstrates **dynamic resizing**, **linear search**, and **element shifting** — core 1D array operations.

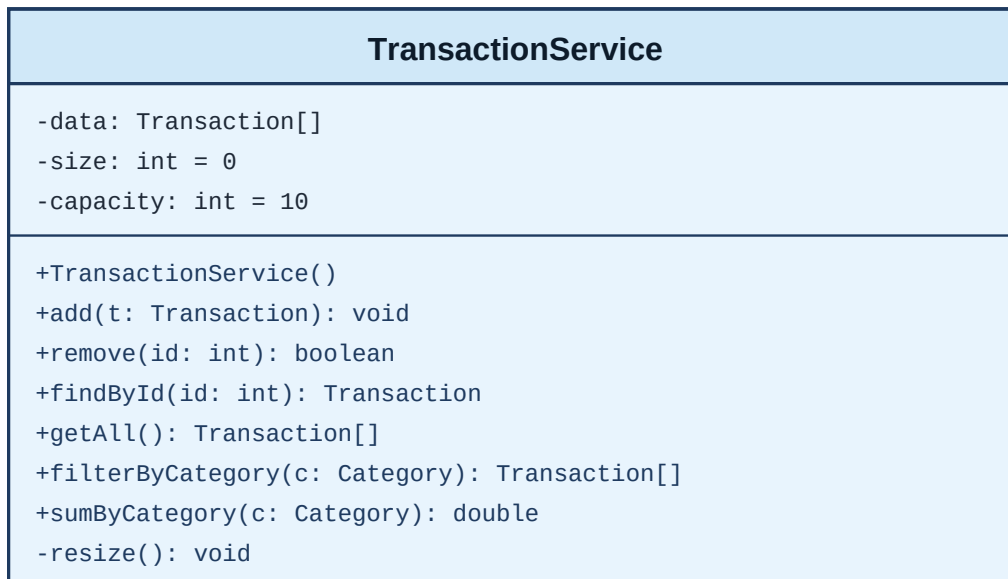


Figure 4f — TransactionService class with 1D array operations.

8.7 BudgetService

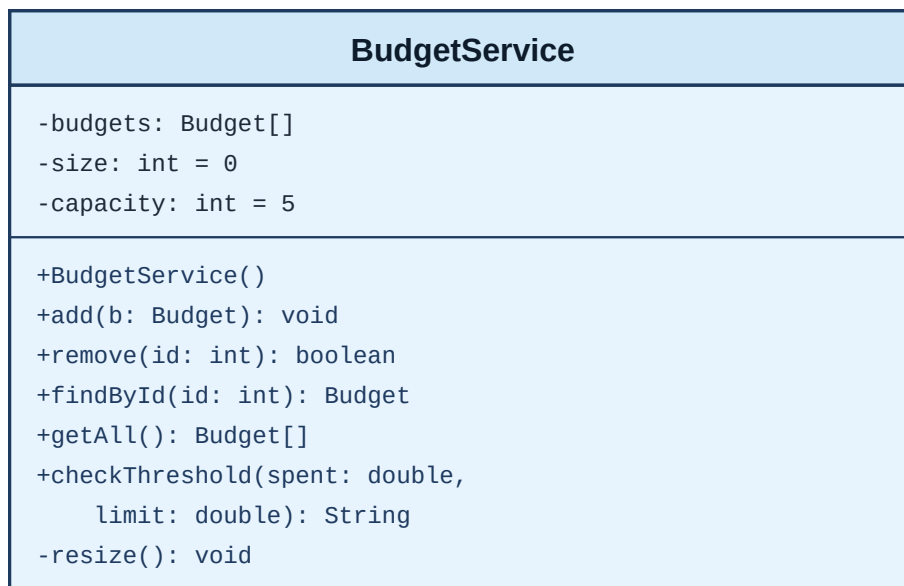


Figure 4g — BudgetService class UML diagram.

8.8 NoteService

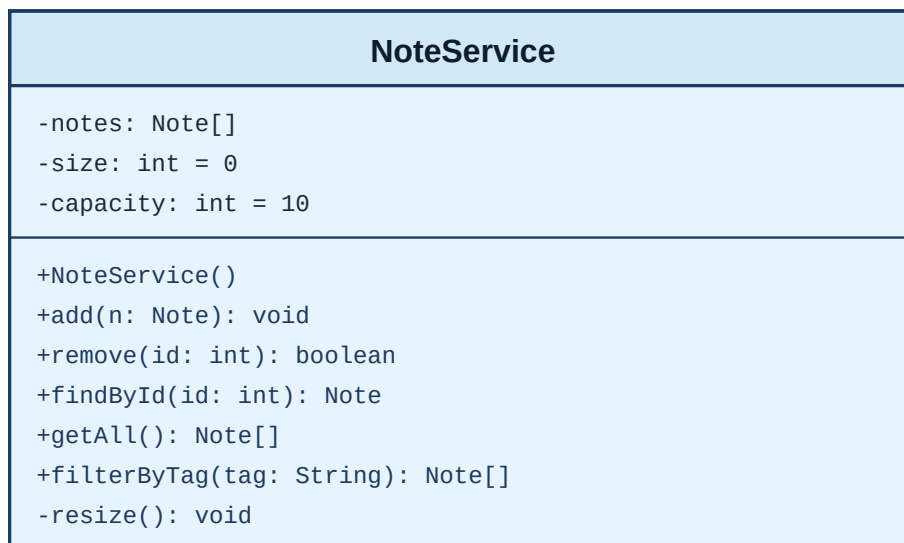


Figure 4h — NoteService class UML diagram.

8.9 IRepository (Interface)

The repository interface enables **Dependency Inversion**. Services depend on this abstraction, not concrete SQLite implementations. This also demonstrates **polymorphism** — any class implementing this interface can be substituted at runtime.

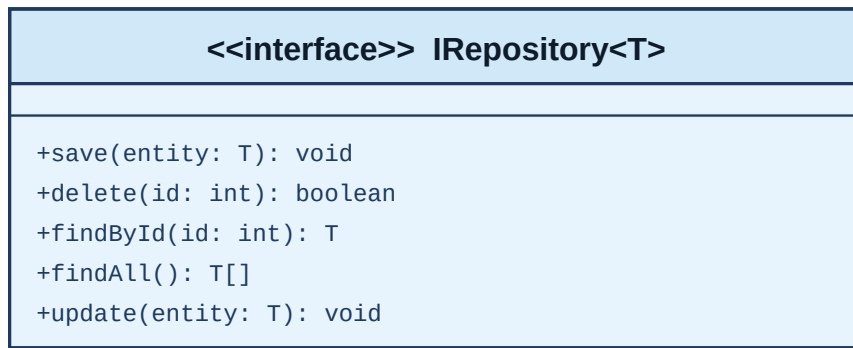


Figure 4i — Generic IRepository interface (Dependency Inversion).

8.10 Class Relationships Summary

Relationship	From	To	Type
Inheritance	Transaction, Budget, Note	BaseEntity	extends (is-a)
Dependency	TransactionService	Transaction[]	uses (has-a)
Dependency	BudgetService	Budget[]	uses (has-a)
Dependency	NoteService	Note[]	uses (has-a)
Implementation	TransactionRepo	IRepository<Transaction>	implements
Implementation	BudgetRepo	IRepository<Budget>	implements
Association	Transaction	Category	uses (enum field)

Array Operations Implemented

- **Add:** Insert at next index; resize (double capacity) when full — dynamic array growth.
- **Remove:** Shift elements left to fill gap — array element shifting.
- **Search:** Linear search by ID — sequential traversal of 1D arrays.
- **List All:** Iterate index 0 to size-1 — standard array iteration.
- **Filter:** Build result array matching a predicate — array filtering pattern.

Sample Code — Array Resizing

```
private void resize() {
    Transaction[] newArr = new Transaction[data.length * 2];
    for (int i = 0; i < size; i++) {
        newArr[i] = data[i]; // copy elements
    }
    data = newArr;
}
```


9 I/O Data Flow Diagram

Traces a typical interaction from input through validation, processing, persistence, and output.

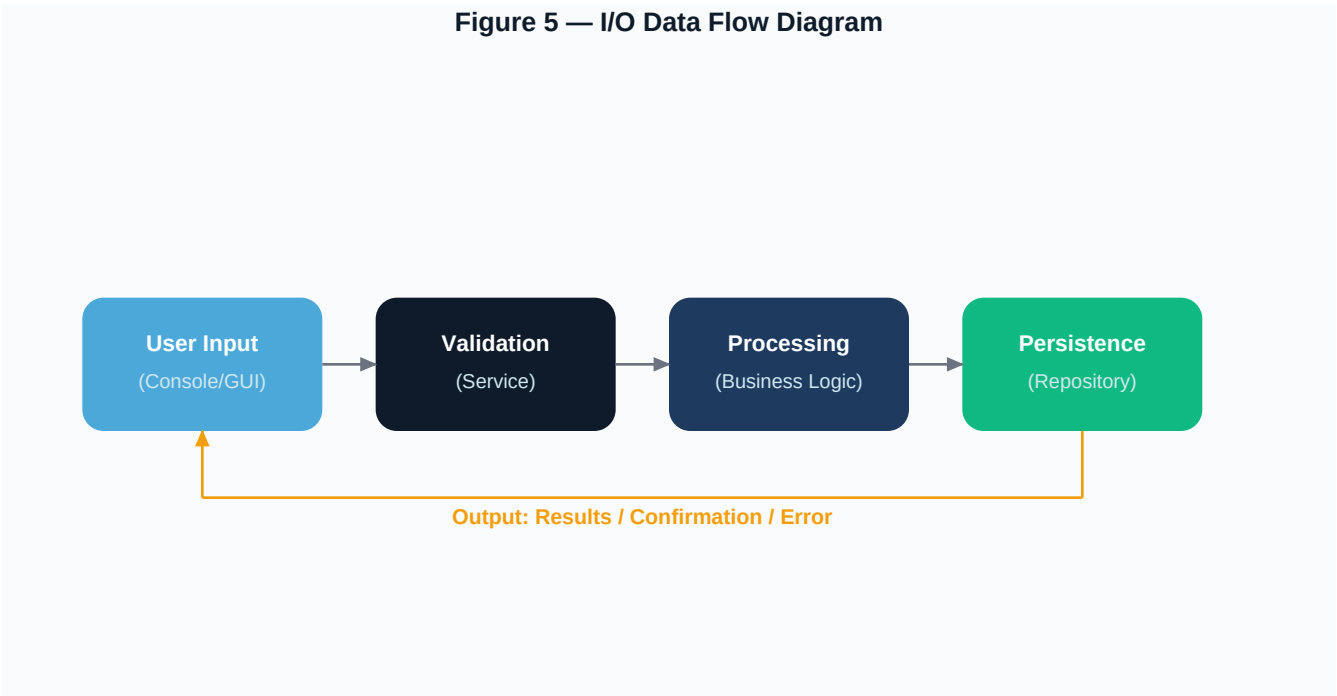


Figure 5 — I/O Data Flow showing the request-response cycle.

10 Database Design

SQLite via JDBC. Three tables mirror the domain model.

Transaction Table

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
name	TEXT	NOT NULL
amount	REAL	NOT NULL, CHECK(amount > 0)
category	TEXT	NOT NULL
date	TEXT	NOT NULL (ISO-8601)

Budget Table

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
name	TEXT	NOT NULL
amount	REAL	NOT NULL, CHECK(amount > 0)

Notes Table

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
title	TEXT	NOT NULL
content	TEXT	
date	TEXT	NOT NULL (ISO-8601)
tag	TEXT	DEFAULT 'general'

11 OOP & Array Design Rationale

This project centres on **core OOP fundamentals** and **1D arrays** rather than higher-level collections. The rationale is educational.

OOP Principles Demonstrated

- **Encapsulation:** All fields private; access via getters/setters.
- **Abstraction:** IRepository hides implementation details.
- **Inheritance:** BaseEntity provides common fields inherited by all models.
- **Polymorphism:** Services use interface references, enabling runtime substitution.

Why 1D Arrays?

- Builds understanding of contiguous memory and index-based access.
- Forces manual implementation of resize, shift, and search.
- Demonstrates performance trade-offs motivating higher-level collections.
- Aligns with CS coursework in data structures and algorithm analysis.
- Prepares for interview questions on array manipulation.

12 User Interface Specification

Console-based menu interface. Future iteration: Java Swing.

Main Menu Structure

[illegible]

Input Validation Rules

Field	Validation Rule	Error Message
Transaction Name	Non-empty, max 100 chars	Name cannot be empty.
Amount	Positive double, max 2 decimal places	Amount must be > 0.
Category	Must match Category enum	Invalid category.
Date	ISO-8601 (yyyy-MM-dd)	Invalid date format.
Budget Limit	Positive double	Limit must be > 0.

13 Future Enhancements

- **Android App:** Kotlin/Java with Room persistence.
- **Web App:** Spring Boot REST API + React frontend.
- **Cloud Sync:** Firebase or Supabase backend.
- **PDF/Excel Reports:** Downloadable financial reports.
- **AI Suggestions:** Spending pattern analysis.
- **Multi-currency:** Currency conversion via API.
- **Collections Migration:** Replace arrays with ArrayList, HashMap, TreeMap.

14 Glossary

Term	Definition
CRUD	Create, Read, Update, Delete — four basic data operations.
POJO	Plain Old Java Object — simple class with fields, getters, setters.
SOLID	Five design principles: SRP, OCP, LSP, ISP, DIP.
3-Tier	Presentation, Business Logic, Data Access separation.
1D Array	Single-dimensional, fixed-size, contiguous data structure.
JDBC	Java Database Connectivity — standard DB access API.
SQLite	Lightweight, file-based relational database engine.
UML	Unified Modeling Language — standard for software diagrams.

15 Revision History

Version	Date	Author	Changes
1.0	May 5, 2026	Zaid	Initial SRS with UML class diagrams.

End of Document